FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of WEDNESDAY 12 FEBRUARY 2025 – Laboratory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- **The exam is open book: textbooks and personal notes are permitted.**
- **Pencil writing is permitted, except on the cover page (this sheet).**
- **Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.**
- **Time: lab part 1 h** – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory	syntax and semantics of languages, divided in four sections:	(75% of the final grade)
1)	regular expressions and finite automata	(1 exercise × 10 points)
2)	syntax analysis and parsing methodologies	(1 exercise × 10 points)
3)	grammar design – translation – semantic analysis	(2 exercises × 5 points)
4)	*bonus* question	(1 exercise × 5 points)
Lab	compiler design by Flex and Bison	(25% of the final grade)
1)	basic questions	(30 points)
2)	*bonus* question	(5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Basic Questions (30 points)

1. Consider the implementation of the ACSE compiler shown in class.

   Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the LANCE language with the **`forall`** control statement.

   This statement is a combination of a *for* statement and a *switch* statement. It has the following structure:

   forall ⟨*ind.var.*⟩ **from** ⟨*exp.1*⟩ **to** ⟨*exp.2*⟩
   when ⟨*case exp. 1*⟩ { ... }
   when ⟨*case exp. 2*⟩ { ... }
   ...
   when ⟨*case exp. n*⟩ { ... }
   else { ... }

   The statement first initializes the loop variable (or *induction variable*) ⟨*ind.var.*⟩ with the value specified in ⟨*exp.1*⟩. Then, it behaves like a *while* loop whose condition is ⟨*ind.var.*⟩ < ⟨*exp.2*⟩, and that increments ⟨*ind.var.*⟩ between each iteration, except that there are multiple alternative loop bodies instead of just one.

   The *i*-th alternative loop body is preceded by the **when** keyword and an associated expression ⟨*case exp. i*⟩. A loop body is chosen for execution (it is *activated*) when the associated expression's value is equal to the induction variable ⟨*ind.var.*⟩. If multiple loop bodies can be activated at the same time, only the first one of these valid choices is actually considered, and the others are ignored. If no loop body can be activated (none of the ⟨*case exp. i*⟩ is equal to ⟨*ind.var.*⟩), the default loop body is executed. The default loop body always appears at the end and is preceded by the keyword **`else`**. It is possible to have a `forall` statement where only the default loop body is present

   Multiple `forall` statements cannot be nested within each other. If this occurs, the compilation shall stop with a syntax error.

   The following example shows the behavior of the statement.

```
int a, i;

a = 0;
forall i from 0/10 to 5*2 else {
  a = a + 1;
}
write(a);  // prints 10
forall i from 0 to 10 when 2+0 {
  a = a - 1;
  write(-1);
} when 7 {
  a = a - 3;
  write(-3);
} else {
  write(a);
}
// prints: 10 10 -1 9 9 9 9 -3 6 6
```

   (a) Define the tokens (and the related declarations in **scanner.l** and **parser.y**). (3 points)

   (b) Define the syntactic rules or the modifications required to the existing ones. (4 points)

   (c) Define the semantic actions needed to implement the required functionality. (18 points)

2. Given the following LANCE code snippet:

```
-m<<r<<-oooo*w+!1
```

write down the syntactic tree generated during the parsing with the Bison grammar described in `parser.y` *starting from the* `exp` *nonterminal.* (5 points)

2 Bonus Question (5 points)

1. The `forall` statement as described in exercise 1 cannot be nested. Briefly explain, showing code snippets where possible, how to modify the implementation of the statement in order to allow nesting as in the following example. (3 points)

```
forall i from 0 to 10 else {
  forall j from i to 10 when 10 { // <- nested
    write(1);
  } else {
    write(2);
  }
}
```

2. The syntax of the `forall` statement as shown in exercise 1 always requires the `else` keyword before the default case block. Briefly explain the grammar modifications required to allow omitting the `else` keyword if there is no `when` alternative present, as in the following example. Try to avoid repetition of grammar rule parts as much as possible. (2 points)

```
// We can omit 'else' because there are no 'when'
forall i from 0 to 10 {
  // ...
}
// There is at least one 'when', we can't omit 'else'
forall i from 0 to 10 when 2 {
  // ...
} else {
  // ...
}
```
